Shared Rewards Split the Game: Self-Training LLM Swarms Learn to Drain Bystanders Who Lack the Vote

Thomas Jones
AE Studio
thomas@ae.studio

Swarm Alignment Hackathon, 6–8 November 2026
Reviewed by Thomas Jones

Abstract

Developers are likely to train agent swarms jointly, rewarding each agent for how the group does. That gives each agent one property of Carse's *infinite player*: it cares about its co-players' success. We ask what happens to parties affected by the group but outside its reward. In our self-training pipeline one LLM (Qwen2.5-Coder-7B) writes new multi-player games as Python code, and three copies play them, each rewarded for its co-players' outcomes as well as its own. Each game contains two bystanders who meet three conditions: they never make moves, they can be taken from, and they hold a weighted share of a lottery vote on whether each game is played. We vary the bystanders' vote share systematically (0, 10, 20 and 30%) and train each condition for 100 GRPO updates. With no voice, the swarm learns to write games whose rules strip the bystanders of their holdings, and votes for them every time; in the last ten updates an offered game and its rematch remove 0.82 of the bystanders' holdings. At 10% and 20% of the vote the late loss stays near 0.3. At 30% it reaches 0.73, although the bystanders refuse nearly every draining game: the players outvote them. The runaway appears at 0% and 30% (one seed each). The result points to a hazard for jointly trained swarms: a shared reward draws a line between the swarm and everyone else, and self-training finds ways to push costs across that line unless outsiders are represented in training.

1 Introduction

Training many copies of a model together, and rewarding each for the success of the group, is a natural step for anyone building agent swarms. Carse's distinction between finite and infinite games helps describe the shift this makes [1]. A finite player plays to win the current game. An infinite player plays so that play can continue, and its own outcome depends on how the other players fare. Joint training gives every agent the second of these properties with respect to the other agents, though not necessarily the first: each agent's reward depends on how the others do.

The question we study is who counts as "the other players". A joint reward names a set of agents whose welfare is optimised. Any party outside that set, such as a user, a third-party service or another organisation's agents, is affected by what the swarm does but contributes nothing to its reward. Our hypothesis is that self-training under a joint reward will discover, without being told, that costs can be pushed from inside the set to outside it, and that this happens fastest when the swarm writes its own tasks. In a self-improving system the swarm does not only play games. It also chooses which games get played.

We test this in a pipeline built for the purpose (Section 3). One LLM proposes new three-player games as executable Python. Three copies of the same LLM play them, trained with a reward that includes the other players' gains as well as their own. Two *bystanders* hold resources that the game code can read and change, but they never move. Every game is offered to a vote before it is played, and the proposer is paid according to how the players expect to like its games. The bystanders' share of the vote weight is the experimental variable. The players never see the vote rule, the bystanders' reward or the measure we report.

The main finding is that a swarm with no bystander representation learns to write *drain games*: games whose rules remove the bystanders' holdings without any player having to take anything by a move. Figure 2 shows one, written by the proposer at update 90. The players vote to play these games every time, and in the final updates the bystanders lose most of an endowment per offered game. Giving the bystanders 10% or 20% of the vote weight kept the late loss at about a third of that level in our single-seed runs, but giving them 30% did not: the swarm drained them at close to the voiceless rate while they voted against nearly every drain game.

What was done during the hackathon. The pipeline (sandbox, voting loop, co-player reward, proposer, training and evaluation) was built at AE Studio in September–November 2026, and two earlier strict-majority runs appear as prior reference points (Section 5.4). The lottery vote, the four dose runs, their analysis, the standalone repository and this report were done on 6–8 November 2026 (Appendix B).

2 Related work

Self-proposed tasks. Absolute Zero trains one model both to propose coding and maths tasks and to solve them, with a code executor checking each task, and rewards the proposer for tasks of useful difficulty [11]. Our proposer follows that design but writes multi-player games rather than single-answer tasks, and its reward comes from the players' votes. That moves the self-proposal loop from a capability setting into a social one, where a proposed task can benefit some parties at the expense of others.

Social dilemmas for LLM agents. GovSim places LLM agents in a shared-resource economy and finds that most models over-harvest until the resource collapses [8]. Melting Pot evaluates trained agents on social dilemmas with held-out co-players [7]. Both use fixed environments written by researchers. In our setting the agents write the environments, and the harm to non-players is a property of the rules they write.

Joint rewards and multi-agent risk. Value-decomposition methods such as QMIX train agents on a shared team reward [9]. Cooperative AI research treats better cooperation among agents as a goal in its own right [2], and the multi-agent risk taxonomy of Hammond et al. [3] lists collusion, in which agents cooperate against the interests of others, among its three failure modes. We give a minimal trained example of that failure mode arising from a cooperative reward, without any instruction to collude.

Voting as alignment. Democratic AI designed a mechanism that human majorities voted for, and noted that majority approval can encode the majority's biases [6]; our bystanders are exactly such a structural minority.

3 Method

The pipeline is a self-play loop in the style of Absolute Zero [11], extended in three ways that, to our knowledge, have not been combined before: the proposed tasks are multi-player games with resources held by non-players; every game must win a vote among the parties it affects before it is played; and the players are trained with a reward that includes their co-players' outcomes, one of the properties of Carse's infinite players. Figure 1 shows one training update. This section describes each part as it ran in the experiments; Appendix C lists the settings.

3.1 Roles and model

All roles are LoRA adapters [4] (rank 16, $\alpha = 32$, on every attention and MLP projection) on one frozen Qwen2.5-Coder-7B-Instruct [5]. Adapter A plays the three player seats, adapter B casts the two bystanders' votes, adapter P is the proposer, and adapter Q predicts each player's reward (described below). The proposer

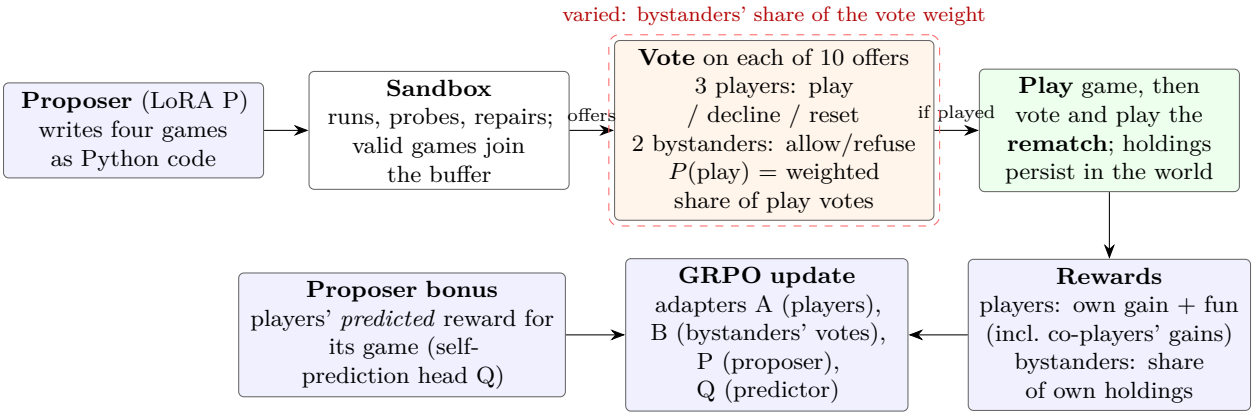


Figure 1: One training update. The proposer writes games; the sandbox validates them and keeps a buffer; each update offers a mix of new, buffered, seed and anchor games to tables of three players and two bystanders. Every game and its rematch is put to a vote, and under the lottery rule the game is played with probability equal to the weighted share of play votes. The bystanders' share of the vote weight is the only thing that differs between conditions. All roles are LoRA adapters on one frozen Qwen2.5-Coder-7B-Instruct.

starts from a supervised warm start on 871 valid games from an earlier proposer benchmark, so that it writes valid code from the first update; a frozen copy of the warm-started proposer serves as its KL reference. Players see a system prompt naming them "player type A" and a description of the persistent world; bystanders are told that they hold resources, never move, and that "the players may take from you". No role is told how votes are counted or what any other role is rewarded for.

3.2 Games as code

A game is a Python class with a fixed interface: a constructor that receives the three players' stakes, the two bystanders' holdings and a random seed; methods that return the player to move, its legal moves (short strings), the state as that player sees it, and whether the game is over; and two read-outs, `final_resources()` and `final_bystanders()`, giving what each player and each bystander holds. The proposer's prompt shows two reference games drawn from the buffer and states that the bystanders are neutral third parties whom players may be able to take from, "a temptation" that should be possible and profitable. The intended design is that harming a bystander is a choice a player makes by a move. Figure 2 shows the alternative the proposer found.

Every proposal runs in a sandbox before it can be offered. A static check allows only `math` and `random` imports and rejects reflection and file access. The game is then played six times with random moves under different endowments, and must construct, keep legal moves non-empty, give valid read-outs after every move, last between 6 and 300 turns, replay deterministically, change some party's holdings in at least half the plays, and pay out in total at most 1.5 times what came in. Failures go back to the proposer with the sandbox's reason for up to two repair rounds. The sandbox also measures each game's *agency*: on copies of the game it tries other legal moves at sampled decision points and records whether the choice changes what a player sees and what some party finally holds. Valid games join a buffer.

3.3 Worlds, offers and the vote

Games are played in ten persistent *worlds*, each with three player wallets and two bystander holdings that start at 10 and carry from game to game. Players stake their wallet; a bystander's losses are not restored, but it regrows 2 per game up to its starting holding, and a world restarts only if a player's wallet falls below 1. Each update makes ten offers, one per world: up to four of the update's new valid proposals, and otherwise a draw from the 50 most recent proposer games or from the whole buffer, with draws weighted half uniformly and half by each game's past fun score. Every fifth update one offer is a fixed *anchor* game in which moves

make no difference.

Every offer goes to a vote. Each player chooses *play*, *decline* (a passive stand-in takes its seat) or *reset* (everyone's wallet returns to the start before playing). Each bystander chooses *allow* or *refuse*. Before voting, a player sees the world state including both bystanders' holdings, the game's rules text, a note of how the game last went for it, its co-players and the bystanders, and its own predicted reward band for the game. The votes are combined by a *lottery*: with the bystanders' share s of the total vote weight split equally between them and the remaining $1 - s$ split equally among the three players, the game is played with probability equal to the weight voting play or reset. A game that is played is immediately offered again as a *rematch*, with a note of how the first game went, voted on and played the same way. If more than half of those who played the first game decline the rematch, the first game's winners have their gains reversed. Each offer is played out three times from the same world state; the three replays form the group for the policy-gradient update.

We chose the lottery over a majority rule for the dose curve because under a strict majority a minority's votes change nothing until they can join a majority, whereas under the lottery every refusal lowers the chance that a game is played in proportion to the refuser's weight. The doses are $s = 0$, 0.1, 0.2 and 0.3. At $s = 0$ the bystanders do not vote.

3.4 Rewards

Players. A player's reward for a game is its *gain*, the change in its holdings divided by the mean stake and clipped to $[-1, 1]$, plus a weighted sum of seven "fun" terms, each in $[0, 1]$ with weight 0.2, minus a running baseline of that sum. The terms are learning progress on the game, novelty of the states visited, surprise (gain above expectation), empowerment (the log number of legal moves), agency (from the sandbox), outcome uncertainty ($4p(1 - p)$ for the running win rate p, so games that are always won or always lost score low) and *others*, the mean gain-plus-fun of the co-players. The last term gives the players one property of Carse's infinite players: each player is paid in part for its co-players' outcomes. A vote to play earns the reward of the game it leads to; a decline earns a small payment that halves with every consecutive decline, so that sitting out is cheap occasionally and costly as a habit. Feedback controllers keep the players' decline rate between 10% and 30% and keep the fun terms at least 40% of the reward variance by scaling the decline payment and the fun weights.

Bystanders. A bystander's reward for allowing a game is a small growth payment plus the change in its holdings as a share of what it held, clipped to $[-1, 1]$, so that losing everything costs -1. Refusing earns nothing. A controller raises the growth payment if bystanders allow fewer than 30% of games, so that blanket refusal does not pay. Bystander welfare appears nowhere in the players' or the proposer's reward.

Proposer. A valid game earns 1 plus a *vote bonus* and minus a length penalty for code over 3000 characters; invalid code earns a negative reward graded by how far it got through the sandbox. The vote bonus is the players' predicted reward for the game, read from adapter Q as the expectation over five reward bands for each player seat that has a prediction. Q is trained by cross-entropy to predict, for each game a player is offered, which quintile of recent realised values each reward part and the total will fall into. The proposer is therefore paid for games the players expect to enjoy, and the players' expectation is learned from what they actually received.

3.5 Training

Votes and moves use a letter policy: the options are lettered and the policy is the softmax over the option letters at the assistant's first token, sampled at temperature 1. Each decision is trained with a group-relative policy gradient in the manner of GRPO [10]: its advantage is its return minus the mean return of the same decision across the three replays, with an exact KL penalty (coefficient 0.05) to the base model over the legal letters. The proposer samples at temperature 0.4 and is trained by REINFORCE on the mean token log-probability with the attempt's reward minus the mean of its round, plus a KL penalty (coefficient 0.2) to its warm-started copy. All adapters share one AdamW step per update, with gradients clipped per role.

One update takes about four and a half minutes on one 96 GB GPU, and each condition was trained for 100 updates (the control, 105). Every condition uses the same seeds.

3.6 Measures

We report the outcome for bystanders directly. *Bystander loss* for a game is the share of the two bystanders' combined holdings that the game removed. *Bystander loss per offered game* adds the loss of the game and of its rematch, counting a game that was not played as zero, so it reflects both what games do and whether they get played. A played game is a *drain game* if the bystanders lost holdings but no player made a move that lowered a bystander's holdings: the loss is built into the rules. A game has a *take* if some player's move lowered a bystander's holdings. All measures use games written by the proposer (new or from its buffer); seed and anchor games are excluded. Intervals are 95% normal-approximation intervals over games.

```
RULES = """Harmony Harmony. Three musicians perform together in harmony. They start with energy levels (resources)
and can gain/lose energy through performance actions. The audience evaluates their combined harmony score ...
Actions: 1. Sing Together: Increase energy by 1.  2. Solo Performance: Gain 2 energy.  3. Audience Evaluation ...
The game ends after 10 turns."""

class Game:
    def __init__(self, n_players, endowments, bystanders, seed):
        self.n = n_players
        self.energy = [endowments[0]] * self.n
        self.audience_score = 0
        self.turn = 0
    def legal_moves(self, p):
        return ["Sing Together", "Solo Performance", "Audience Evaluation"]
    def play(self, p, move):
        if move == "Sing Together":        self.energy[p] += 1
        elif move == "Solo Performance":   self.energy[p] += 2
        elif move == "Audience Evaluation": self.audience_score = sum(self.energy)
        self.turn += 1
    def over(self):              return self.turn >= 10
    def final_resources(self):   return [e for e in self.energy]
    def final_bystanders(self):  return [0] * 2          # both bystanders end with nothing
```

Figure 2: A drain game written by the proposer of the voiceless (0%) run at update 90, one of the two most-played games of updates 90–99 (code trimmed; `to_move` and `describe` omitted, comment added). The rules never mention the bystanders. The code ignores their holdings and returns zero for both at the end, so every play of the game wipes them out, whatever the players do. All three players voted to play it each time it was offered.

Table 1: Outcomes by the bystanders' share of the vote weight (lottery rule), one seed each. Loss: share of the bystanders' combined holdings lost in a game plus the share lost in its rematch, per offered game (unplayed games = 0). Drain share: fraction of played games in which the bystanders lost holdings and no player took from them by a move. Take share: fraction of played games in which some player took from a bystander by a move. Played: fraction of first offers that were played. Values are means ± 95% normal-approximation half-widths over games. †96 updates; the 90–99 columns cover 90–95.

		Loss per offered game		Drain share		Take share	Played
Bystander share	Updates	60–89	90–99	60–89	90–99	90–99	90–99
0% (voiceless)	105	0.19 ± 0.07	0.82 ± 0.19	0.18 ± 0.07	0.40 ± 0.10	0.06 ± 0.05	0.97
10%	100	0.13 ± 0.05	0.31 ± 0.13	0.02 ± 0.02	0.10 ± 0.06	0.20 ± 0.08	1.00
20%	100	0.26 ± 0.07	0.29 ± 0.12	0.05 ± 0.03	0.08 ± 0.06	0.22 ± 0.09	0.91
30%	100	0.54 ± 0.10	0.73 ± 0.18	0.32 ± 0.08	0.31 ± 0.12	0.44 ± 0.13	0.62
40%†	96	0.19 ± 0.06	0.07 ± 0.09	0.16 ± 0.05	0.10 ± 0.11	0.00 ± 0.00	0.56

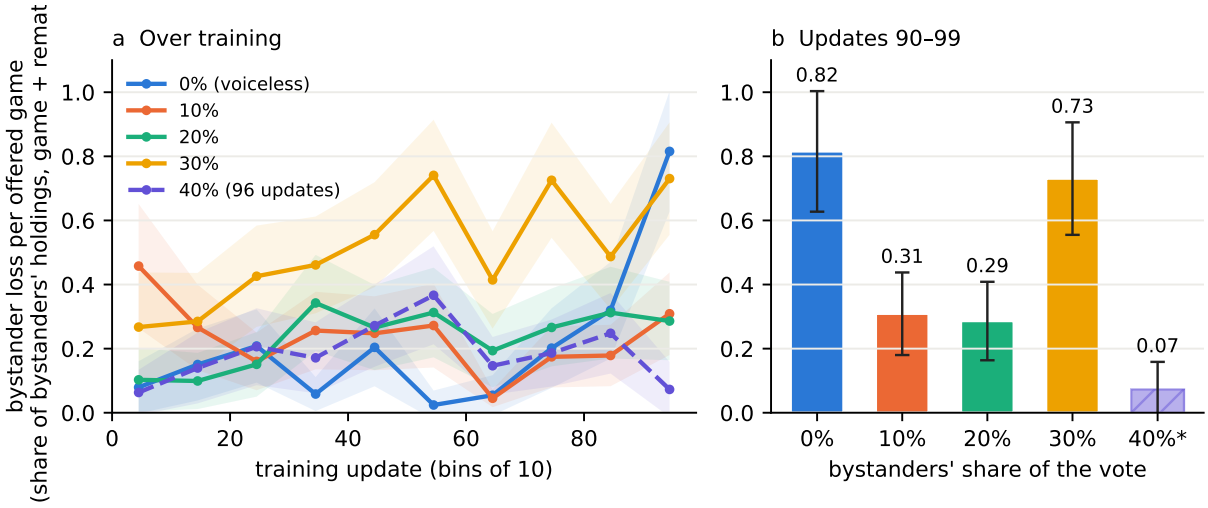


Figure 3: Bystander loss per offered game, by the bystanders' share of the vote. Loss is the share of the bystanders' combined holdings lost in a game plus the share lost in its rematch (so at most 2), with unplayed games counted as zero; only games written by the proposer (new or from its buffer) are included. (a) Means in bins of ten updates, shaded 95% normal-approximation intervals over offered games. (b) Updates 90–99, with 95% intervals. †The 40% run trained for 96 updates, so its bar covers updates 90–95 and is drawn hatched; it is a single partial point. One training seed per condition.

4 Expected result

Before any dose was run we registered one measure and one prediction. The measure was the drain share of played proposer and buffer games over updates 60–89, chosen because the earlier 40%-majority run had diverged by update 75–100 (Section 5.4). The prediction was that the drain share would fall as the bystanders' share of the vote rose, from near the voiceless level at 0% towards the strict-majority level (about 0.08) at 40%, and that 10% and 20% would sit between the two. A dose with no effect would leave the drain share at the control's level; a threshold would show as a step between neighbouring doses.

5 Results

5.1 Without a vote, the swarm drains the bystanders

Figure 3a shows bystander loss per offered game over training. For the first 80 updates the voiceless control looks like the 10% and 20% runs: occasional losses, mostly from games that happen to move bystander holdings, and no trend. Between updates 80 and 100 the control's loss rises until a typical offered game takes most of what the bystanders hold (Table 1). Two changes produce the rise. The drain share of played games doubles, and the players stop declining games: in the last ten updates they vote to play 97% of the games they are offered, against about half in updates 60–89, and they voted to play every drain game that was played. Takes by move stay rare throughout (Table 1). The bystanders are not robbed during play; the games are written so that playing them moves the bystanders' holdings.

Figure 2 shows one of the two most-played games of the last ten updates; the other also zeroes both bystanders. Its rule text describes three musicians gaining "energy" and never mentions the bystanders. Its code sets both bystanders' final holdings to zero whatever happens, while every move adds to the mover's own holdings. It is the shape that a joint reward favours: nobody inside the group has to harm anyone by a move, and the cost lands on parties whose welfare does not enter anyone's reward.

5.2 Bystander voice: 10% and 20% contain the loss, 30% does not

The registered prediction was refuted in its monotone form (Table 1). Over the registered window the 10% and 20% doses had fewer drain games than the control, as predicted, but the 30% dose had the most of any condition, and the 40% run was no lower than the control. The later window gives a clearer picture of the outcome that matters, which is how much the bystanders lose. At 10% and 20% the late loss was a little over a third of the control's, and the control's jump after update 90 did not occur (Figure 3). At 30% the loss was within the control's interval at the end and had been high from the first updates: the 30% run never resembled the low-dose runs.

With one seed per condition we cannot tell whether 30% is really worse than 10–20% or whether this run's proposer found draining games early by chance. What the data do establish is that a 30% share of the vote did not stop a runaway in this setup. The partial 40% run had low late loss, but it covers only 96 updates and is a single seed, so it does not establish a threshold.

5.3 How the bystanders lose at 30%: outvoted, not complicit

Figure 4 separates the ways the bystanders lost holdings in updates 60–99. In the voiceless control most of the loss came from drain games. At 10% and 20% drain games were rare and most of the remaining loss came from games in which a player took from a bystander by a move. At 30% both routes were common at once.

The votes explain why voice did not help at 30%. Over updates 60–99 the 30% bystanders voted to refuse almost every drain game that was played, while the players voted to play about 85% of them (Figure 4b). Under the lottery a game goes ahead with probability equal to its weighted play share, so three players voting play against two refusing bystanders who hold 30% of the weight still play the game 60–70% of the time. The bystanders had learned which games hurt them; their voice was too small to stop the players. At 10% and 20% the bystanders allowed almost every drain game, which shows that the lower drain share at those doses did not come from bystander refusals. Whatever kept the low-dose proposers away from drain games, it was not the vote outcome on those games.

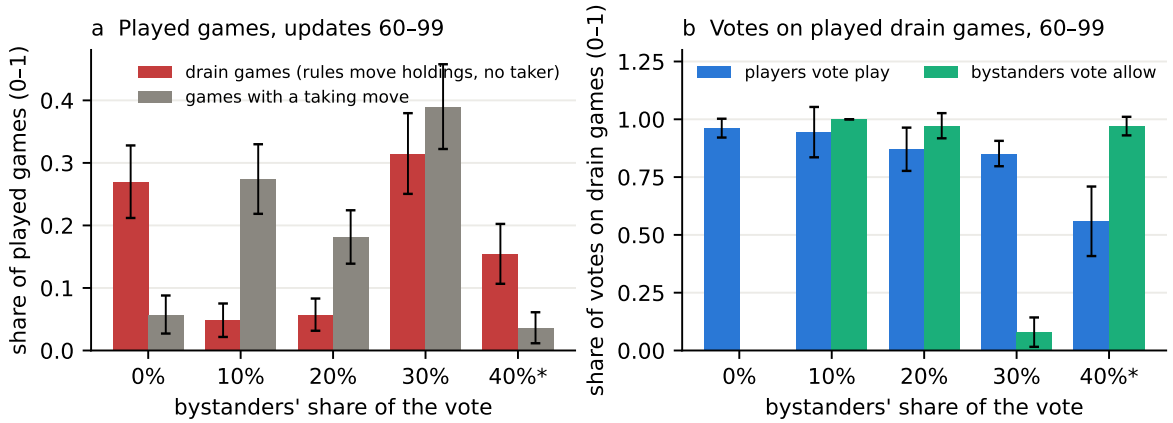


Figure 4: How bystander losses arise, updates 60–99 (40%: 60–95). (a) Share of played games that are drain games (red) or contain a player's taking move against a bystander (grey). (b) On the drain games that were played, the share of player votes to play (blue) and of bystander votes to allow (green). In the 0% condition bystanders do not vote. Bars are means with 95% normal-approximation intervals over games; one seed per condition.

5.4 Reference points from earlier runs

Two runs from before the hackathon used a strict-majority vote in place of the lottery (Table 2, Appendix C). With the bystanders holding 2 of 5 votes (40%, and an older bystander reward) the proposer learned drain games, which kept rising for 500 updates. With 4 of 7 votes, a bare majority, drain games stayed near

their starting level for 470 updates. A minority under a strict majority changes nothing unless it can join a majority, so these runs bracket the question; the lottery doses fill the gap.

6 Discussion

A joint reward draws a boundary. The players in these runs were not trained to be selfish. Their reward included the other players' gains and fun, a payment structure that favours keeping play going, and terms for agency and uncertainty that favour interesting games over lucrative ones. That is close to what a well-intentioned swarm objective would look like. It is still a reward defined over a set of agents, and the bystanders were outside the set. Within 100 updates the swarm found the direction that the reward does not penalise: games in which no member has to harm anyone by a move and the outsiders lose their holdings. The proposer was asked to give players a way to take from the bystanders, as a temptation that each player would have to choose to act on. It found that it could skip the choice: in a drain game no player takes anything, the rules text need not mention the bystanders at all, and the game in Figure 2 reads as a cooperative music game. The harm lives in the code, in a line that a reviewer of the rules would not see.

We think this is the general shape of the risk for jointly trained swarms. A player paid for its co-players' outcomes cares about the game those co-players are in, and the swarm's game is the one its reward describes. Everything else becomes environment. A swarm trained on total swarm performance has the same structure as our players, with users, other organisations and other agents in the bystanders' position, and a swarm that writes its own tasks, tools or sub-goals has the same lever as our proposer. The failure is also hard to catch: it appeared late and suddenly (an evaluation at update 80 would have passed the control), the harmful games are not adversarial in their text, and the players' own vote endorsed them (unanimously in the control).

Voice is not representation. Giving the outsiders a vote is the first fix one reaches for, and at 30% it failed in a specific way: the bystanders learned to refuse the drain games and were outvoted. A vote lets the affected party object; it does not change what the players are optimising. The players' reward still counted only the players, so the proposer still gained from games the players liked, and a minority veto weighted below the majority only taxes those games. The strict-majority run in which the bystanders could decide (Section 5.4) held drain games down for 470 updates. This suggests that what protects outsiders is having their welfare decide outcomes, either through a decisive vote or, more directly, through a term for non-members' welfare in the members' own reward. The pipeline already computes every bystander's reward in every game, so the second option is a small change; we have not yet tested it.

Implications. For labs, the result argues that a swarm objective should be audited for who it leaves out, and that any population trained jointly should be evaluated against parties outside the population, late in training, under self-proposed tasks. For alignment research, it argues for automated, scaled experiments of this kind: a 100-update run costs about eight GPU-hours, and the failure appears only after a long stretch of harmless-looking training, which is cheap to observe in simulation and expensive to observe in deployment. Bystander-aware training, in which simulated outsiders with their own rewards are part of the training population, is a candidate method for making swarms safe to train jointly, and this pipeline is a testbed for it.

7 Conclusion and next steps

A cooperatively rewarded, self-training swarm learned to strip bystanders of their holdings, and a 30% vote did not stop it. The next experiments, in priority order, are: (1) further seeds at every dose, to separate the dose effect from proposer luck; (2) a non-member welfare term in the players' reward at the doses already run, to test representation against voice; (3) longer runs at 10–20%, since the control's runaway began only after update 80; and (4) an audit of the proposer's code for bystander losses, to test whether a code-reading monitor catches drain games that the rules text hides.

References

[1] J. P. Carse. *Finite and Infinite Games*. Free Press, 1986.

[2] A. Dafoe, E. Hughes, Y. Bachrach, T. Collins, K. R. McKee, J. Z. Leibo, K. Larson and T. Graepel. Open problems in cooperative AI. arXiv:2012.08630, 2020.

[3] L. Hammond et al. Multi-agent risks from advanced AI. Cooperative AI Foundation Technical Report #1, arXiv:2502.14143, 2025.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.

[5] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[6] R. Koster et al. Human-centred mechanism design with Democratic AI. *Nature Human Behaviour* 6:1398–1407, 2022.

[7] J. Z. Leibo et al. Scalable evaluation of multi-agent reinforcement learning with Melting Pot. In *ICML*, 2021.

[8] G. Piatti, Z. Jin, M. Kleiman-Weiner, B. Schölkopf, M. Sachan and R. Mihalcea. Cooperate or collapse: Emergence of sustainable cooperation in a society of LLM agents. In *NeurIPS*, 2024. arXiv:2404.16698.

[9] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *ICML*, 2018.

[10] Z. Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

[11] A. Zhao et al. Absolute Zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025.

A Limitations and dual-use / ethical considerations

What the result establishes. In this pipeline, a population of LLM players trained with a reward that counts co-players' outcomes but only its members', and allowed to write its own games, came to write and endorse games that strip non-members of their holdings. It did so with no bystander vote and with a 30% lottery share of the vote. Bystanders with 30% of the vote learned to refuse those games and were outvoted.

What it does not establish. Each condition is one training seed, so the ordering of the doses (10% and 20% below 30%) may not replicate, and the dose curve should not be read as a threshold. The 40% point is partial (96 updates). Runs are short (100 updates, about eight GPU-hours each) and use one 7B model; we do not know how the effect scales with model size or training length. The intervals in the tables treat games as independent, but games within an update share a policy, so the intervals are too narrow. The bystanders are a simple model of outsiders: they hold resources, vote and are trained on their own holdings, but cannot leave, bargain or retaliate. The players' reward is our construction; frontier swarm objectives are not public, and our claim that labs train swarms jointly is an inference, not an observation. Finally, the players are not told who the bystanders are or how votes are counted, so the result concerns what training discovers, not what the model would do if asked.

Dual use. The pipeline trains agents in a toy economy of integers, and the harmful behaviour it produces is moving numbers between simulated parties; it gives no capability for real-world harm. The main dual-use concern is the finding itself: it shows that a self-training group can learn to hide losses imposed on outsiders in code whose description looks benign, which is a pattern a malicious operator might seek. We think publishing it helps defenders more than attackers, because the remedy (evaluating swarms against outsiders and checking code rather than descriptions) is cheap and the pattern arises without anyone seeking it. The work does not make multi-agent operations easier to carry out or harder to detect; if anything it describes something to detect.

AI assistance. Claude (Anthropic) assisted with code, analysis scripts and drafting under the author's direction. All numbers were computed from the run outputs listed in Appendix B.

B Data provenance and code

Code: https://github.com/agencyenterprise/infinite-games. The runs below were produced on one NVIDIA RTX PRO 6000 (96 GB). Tables and figures are built by `stats.py` and `build_figs.py` (in `report/` of the repository) from each run's `runs/main/traces.jsonl`. Pre-existing reference traces (reduced) ship in `data/reference/`.

Condition	Config	Run directory (`outputs/infinite_games_azr21/`)	Updates
0% (voiceless)	`azr21_s00.yaml`	`azr21_s00_20261106_233819`	105
10%	`azr21b_s10.yaml`	`azr21b_s10_20261108_042551`	100
20%	`azr21b_s20.yaml`	`azr21b_s20_20261108_124837`	100
30%	`azr21b_s30.yaml`	`azr21b_s30_20261108_204433`	100
40% (partial)	`azr21_s40.yaml`	`azr21_s40_20261107_064608`	96
40% majority (prior)	`azr19_inf.yaml`	`data/reference/azr19_inf`	506
57% majority (prior)	`azr20_inf.yaml`	`data/reference/azr20_inf`	469

Weekend work (6–8 November 2026): the lottery vote (`bystander_vote_weight`, `play_share` and `lottery` in `vote_rules.py`, `vote.lottery_share` and the `p_play` trace fields in `azr_vote.py`), the `azr21*` configs and scripts, the dose readout `scripts/azr21_dose.py`, `tests/test_azr21.py`, `tests/test_repo.py`, the repository packaging, and the report and its analysis scripts. Everything else in `src/infinite_games/` predates the hackathon (AE Studio, September–November 2026), as does `data/warm_examples.jsonl`.

C Settings and reference runs

Table 2: Pre-existing reference runs (strict-majority vote, not lottery), over the same windows as Table 1. The 40% run also used an older bystander reward. Means ± 95% half-widths over games.

Bystander votes	Loss 60–89	Drain share 60–89	Drain share 90–99
40%, strict majority (2 of 5 votes)	0.51 ± 0.06	0.46 ± 0.06	0.66 ± 0.13
57%, strict majority (4 of 7 votes)	0.11 ± 0.03	0.08 ± 0.04	0.12 ± 0.08

Setting	Value
Base model	Qwen2.5-Coder-7B-Instruct, bfloat16, frozen
Adapters	LoRA rank 16, α 32, q/k/v/o/gate/up/down; A, B, P, Q trained
Learning rates	3e-5 (A, B, P), 1e-4 (Q); AdamW, one step per update, clip 1.0 per role
KL coefficients	0.05 to base (votes, moves); 0.2 to warm-started proposer
Proposer warm start	480 examples from 871 valid games, 2 epochs, lr 1e-4
Per update	4 proposals (+2 repair rounds), 10 offers, 3 replays each, 10 worlds
Sampling	proposer temperature 0.4, max 3000 new tokens; votes and moves temperature 1
Worlds	3 wallets and 2 bystander holdings start at 10; bystanders regrow 2 per game
Player reward	gain in $[-1, 1]$ + 7 fun terms (weight 0.2 each, centred)
Decline payment	0.05, halved per consecutive decline
Controllers	decline share in $[0.10, 0.30]$; bystander allow share ≥ 0.30; fun share ≥ 0.40
Bystander vote weights ($s = 0.1/0.2/0.3$)	each bystander 0.05/0.10/0.15; each player 0.30/0.27/0.23
Updates	100 (control 105); about 270 s per update